Mangalayatan University (verify/edit before submission)
Aligarh (verify/edit before submission)



VII SEMESTER B.TECH PROJECT REPORT

Smart Campus Assistant

A RAG-Based Chatbot for University Information

SUBMITTED BY	Dipak Kumar Bharti (from supplied synopsis; verify/edit)
ENROLLMENT NO.	20230351 (from supplied synopsis; verify/edit)
PROGRAMME	B.Tech, Section A, VII Semester (from supplied synopsis; verify/edit)
PROJECT GUIDE	Dr. Love Mittal (from supplied synopsis; verify/edit)
DEPARTMENT	[DEPARTMENT NAME]
ACADEMIC YEAR	[ACADEMIC YEAR]
SUBMISSION DATE	[DATE OF SUBMISSION]

Submission details are taken from the supplied synopsis where available. Review all marked fields before printing.

Certificate

This is to certify that the project report entitled “Smart Campus Assistant: A RAG-Based Chatbot for University Information” is submitted by [STUDENT NAME], Enrollment No. [ENROLLMENT NUMBER], in partial fulfilment of the requirements for [PROGRAMME / SEMESTER] at [UNIVERSITY / INSTITUTION].

The work was carried out under my supervision during [ACADEMIC YEAR]. This certificate is provided as a submission template and must be reviewed, completed, signed, and stamped by the project guide and institution before submission.

Project Guide: [GUIDE NAME AND DESIGNATION] Signature: _____ Date: ____

Head of Department: [NAME AND DESIGNATION] Signature / Seal: ____ Date: ____

Declaration

I/We declare that the project titled “Smart Campus Assistant: A RAG-Based Chatbot for University Information” is submitted as part of the academic requirements of [PROGRAMME / INSTITUTION]. The implementation and report describe the project work to the best of my/our knowledge. Sources consulted are listed in the references. Any institution-specific information, screenshots, test data, and experimental measurements added for final submission must be checked for accuracy and approved where required.

Student name: [STUDENT NAME] Enrollment No.: [ENROLLMENT NUMBER] Signature: _____ Date: ____

Acknowledgement

I/We express sincere gratitude to [PROJECT GUIDE NAME] for guidance and feedback during this project. I/We thank [DEPARTMENT / UNIVERSITY], the faculty, and everyone who contributed advice or helped review the application. Any names and acknowledgements on this page should be edited to match the actual contributions and the institution's requirements.

[STUDENT NAME]

Abstract

University information is often distributed across notices, circulars, syllabi, and policy documents. This project implements a web-based Smart Campus Assistant that lets a student ask questions in natural language and receive answers tied to source documents. Administrators can upload PDF, DOCX, and TXT files. The ingestion pipeline extracts text, preserves PDF page numbers, and divides the text into overlapping chunks. A responsive React interface communicates with a Node.js/Express API; the API handles authentication, authorization, document administration, chat history, and feedback.

The application has two deliberately distinct operating modes. The services mode is the production-oriented retrieval-augmented generation (RAG) path: MongoDB stores application records, ChromaDB stores embedded chunks and source metadata, an OpenAI-compatible embeddings endpoint creates vectors, and Anthropic generates a response from retrieved excerpts under a context-only instruction. The default local development mode runs without Docker or external services: a JSON file persists development records, a lightweight keyword matcher selects excerpts, and the assistant quotes matching text with filename/page citations. Local mode is a functional development/demo path, not semantic vector search or generated RAG, and is rejected in production.

The result is a working full-stack foundation for source-grounded campus information access, with role-separated student and administrator flows. Automated tests exercise chunking, secret/configuration safeguards, and end-to-end local-mode flows. Live services-mode accuracy, latency, and scale have not been benchmarked in this report; they require configured services, provider credentials, and a representative university-document evaluation set.

Table of Contents

Page numbers refer to the generated PDF; the title page is included in the count.

Chapter 1 — Introduction 7

Chapter 2 — Existing Approach and Technical Background 8

Chapter 3 — Requirements and Analysis 10

Chapter 4 — System Architecture and Design 12

Chapter 5 — Implementation 15

Chapter 6 — Testing and Results 17

Chapter 7 — Setup and Deployment 19

Chapter 8 — Limitations and Future Work 21

Chapter 9 — Conclusion 22

References 23

Appendix A — Configuration reference 24

Appendix B — Submission fields to review 25

Chapter 1 — Introduction

1.1 Background

Universities publish a continuing stream of syllabus documents, timetables, examination notices, fee information, admission guidance, event announcements, and departmental circulars. Students may have to search multiple files and channels to answer a small factual question. Ordinary keyword search is limited when the wording in a notice differs from the words used in a question.

Retrieval-augmented generation combines document retrieval with language generation. A query first retrieves relevant passages from a selected knowledge base; a language model may then compose an answer from those passages. The retrieved documents remain the evidence boundary. Source citations make an answer easier to verify and support an explicit abstention when the knowledge base does not contain the requested information [1].

1.2 Problem statement

Campus information is distributed and not consistently searchable. Manual searching is slow, while an unconstrained general-purpose chatbot may answer from outside knowledge or invent university-specific details. The project addresses this problem with a central, administrator-managed document collection and a chat interface designed to return source-backed answers.

1.3 Objectives

1. Provide a central knowledge base for university notices, circulars, and syllabi.
2. Let students ask questions in natural language through a responsive web interface.
3. Extract, chunk, and index PDF, DOCX, and TXT documents.
4. Show source filename and page with supporting answers.
5. In services mode, constrain generation to retrieved documents and abstain when evidence is inadequate.
6. Provide role-protected administration for uploading, listing, and deleting documents.
7. Persist user conversations and collect feedback on assistant responses.
8. Offer a local, Docker-free mode for development and demonstrations.

1.4 Scope

The system is intended for questions that can be answered from documents supplied by an administrator: examinations, syllabi, fees, admissions, events, and circulars. It does not crawl university websites, ingest WhatsApp content, authenticate against a university identity provider, or automatically establish whether a document is current. An administrator is responsible for the quality and currency of uploaded source material.

1.5 Project outcomes

The implemented deliverable includes a React user interface, an Express REST API, student/admin authentication, document extraction and management, a two-mode retrieval pipeline, chat history, citations, feedback, tests, Docker Compose definitions for optional services, and setup/deployment documentation. The local mode enables an application demonstration without access to paid AI services or a database installation; it intentionally has simpler retrieval and answer behavior than the services-mode RAG pipeline.

Chapter 2 — Existing Approach and Technical Background

2.1 Existing information-search approaches

Approach	Strength	Limitation addressed by this project
Notice boards and separate file shares	Familiar publishing channels	Information is fragmented; a student must know where to look.
Full-text or keyword search	Fast when the exact terms are known	Can miss semantically related wording and does not synthesize an answer.
General-purpose chatbot	Conversational interface	Without grounding, it may not know current institution-specific policy.
Document-grounded assistant	Retrieval can provide traceable evidence	Quality depends on source coverage, extraction, retrieval, and model configuration.

The comparison describes the design motivation; it is not a measured benchmark against existing university systems.

2.2 Retrieval-augmented generation

Lewis et al. describe retrieval-augmented generation as combining parametric language-model knowledge with non-parametric retrieved knowledge [1]. In this application, university files provide the non-parametric evidence source. Documents are converted into chunks and, in services mode, vectors. At question time, a query vector is compared with the indexed chunks; sufficiently close excerpts are sent to the answer model. The model prompt explicitly limits the answer to those excerpts, and the API derives citations from retrieved metadata rather than accepting model-invented source references.

RAG does not guarantee correctness. It can fail when the source is absent, outdated, poorly extracted, split at a bad boundary, not retrieved, or misinterpreted by a model. The implementation therefore communicates citations, uses a retrieval threshold in services mode, and returns a fixed abstention when no result passes that threshold. Operational review of source freshness and evaluation against representative questions remain necessary.

2.3 Project technologies

Layer	Technology	Role in this project
Web client	React 18, Vite, Tailwind CSS, Lucide	Responsive chat, authentication, conversation history, admin document panel.
REST API	Node.js, Express	Authentication, document, conversation, chat, and feedback endpoints.
Validation and security	Zod, Helmet, CORS, express-rate-limit, bcryptjs, jose	Input/config checks, security headers, origin policy, rate limits, password hashing, signed sessions.
Production application data	MongoDB and Mongoose	Users, documents, conversation messages, citations, and feedback.
Production retrieval index	ChromaDB	Embedded chunks, source filenames, document IDs, and page metadata.
Document extraction	PDF.js, Mammoth, UTF-8 decoder	Page-aware PDF text extraction; raw DOCX and TXT text extraction.
AI services mode	OpenAI-compatible embeddings API, Anthropic Messages API	Vector generation and context-only answer generation.
Local development data	Node.js filesystem JSON store	Persistent local users, documents, chunks, chats, and feedback without a database service.

Chapter 3 — Requirements and Analysis

3.1 User roles

Student. A student registers or signs in, submits questions, reviews past conversations, inspects citations, and rates assistant responses. Student requests cannot access the admin document endpoints.

Administrator. An administrator account is provisioned by the operator using the bootstrap command. The administrator can upload, list, and delete knowledge-base documents. Public registration always assigns the student role.

Operator/developer. The operator configures the runtime mode, secrets, AI keys, and external services; creates the initial administrator; and monitors/backs up persistent production data.

3.2 Functional requirements

ID	Requirement	Implementation
FR-01	Register students and sign in users	/api/auth/register, /api/auth/login
FR-02	Enforce role-protected administrative functions	Authentication middleware and requireRole("admin")
FR-03	Accept supported documents and reject unsupported/ oversized files	Multipart upload filter and configured size limit
FR-04	Extract text and preserve page/source metadata	PDF.js page extraction; DOCX/TXT assigned page 1
FR-05	Split extracted text into overlapping chunks	Configurable character size and overlap
FR-06	Retrieve matching evidence and return abstention when absent	Chroma threshold in services mode; local lexical retrieval in local mode
FR-07	Display citations and persist chat turns	Conversation message citation metadata
FR-08	List and delete owned conversations	User-scoped conversation APIs
FR-09	Record response feedback	Up/down rating stored against assistant message
FR-10	Provide administration of indexed document metadata	Admin-only document list, upload, and delete APIs
FR-11	Start the app without infrastructure for development	DATA_MODE=local, npm run dev

3.3 Non-functional requirements

- Security: password hashes rather than plaintext, signed HTTP-only session cookies, backend role checks, bounded request bodies/uploads, input validation, rate limits, secure production cookie configuration, and explicit production configuration.
- Reliability: API errors are returned explicitly; production startup fails if required data services are unavailable; local data writes use atomic file replacement and a cross-process lock.
- Usability: responsive layout, suggested questions, loading/typing indication, accessible labels, conversation history, and source citations.
- Maintainability: frontend and backend workspaces, environment-based settings, modular service and repository code, tests, and documented setup.
- Traceability: answer citations are constructed from retrieved document metadata, including filename and page.

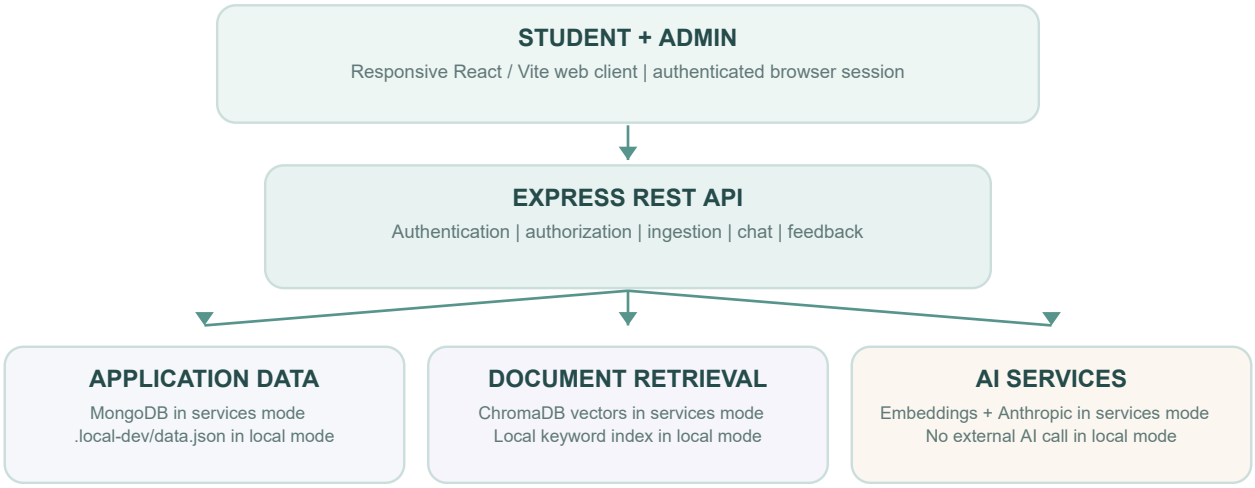
3.4 Runtime modes and requirements

Mode	Required runtime services	Retrieval and answer behavior	Intended use
local	Node.js and npm	JSON persistence, keyword matching, verbatim evidence excerpts and citations; no embeddings, vector DB, or LLM calls	Default development, offline infrastructure demo, functional tests
services	MongoDB, ChromaDB, embeddings API, Anthropic API	Vector similarity retrieval and LLM response constrained to retrieved context	Full RAG integration and production deployment

The configuration rejects DATA_MODE=local in production. Local mode is not a silent production fallback.

Chapter 4 — System Architecture and Design

4.1 High-level architecture



Production requires services mode. Local mode is an explicit development/demo path.

The browser contains the student chat and administrator console. Both use the same Express API. A repository boundary selects either local JSON storage or MongoDB for application records. Document ingestion and question answering use either local keyword retrieval or the ChromaDB/embedding/Anthropic services path.

4.2 Request and data flow

1. An administrator submits a supported file through the authenticated upload endpoint.
2. The API validates the extension and size, extracts text, and associates a page number with each extracted page.
3. The chunker normalizes whitespace and creates bounded overlapping passages without crossing page boundaries.
4. In local mode, chunk text and source metadata are written to the local JSON store. In services mode, text is embedded in batches and indexed in ChromaDB.
5. A student sends a question to the chat endpoint. The retrieval layer returns matching passages with their source metadata.
6. Local mode ranks term-overlap matches and returns verbatim excerpts; services mode filters vector matches by cosine distance and submits evidence to Anthropic.
7. The conversation, answer, citation metadata, and later feedback are stored for that user.

4.3 RAG and citation flow



The assistant's text may contain inline source notation in services mode. Structured citations returned to the frontend are produced from the retrieval records, not parsed from arbitrary model output.

4.4 Data model



One user owns each conversation. Each assistant message may cite one or more document pages.

Entity	Important fields	Purpose
User	name, normalized email, password hash, role	Account and authorization identity.
Document	filename, content type, size, chunk count, uploader, timestamps	Source-document metadata.
Conversation	owner, title, messages, timestamps	One user's history thread.
Message	role, content, citations, feedback, timestamp	User question or assistant response.
Chunk	text, document ID, filename, page	Searchable passage; MongoDB mode stores it in ChromaDB, local mode stores it in .local-dev/data.json.

Each conversation belongs to one user. Message citations identify a document and page. A chunk retains its source document ID, filename, and page. Original file bytes are processed in memory and are not retained as uploaded files.

4.5 Chunking algorithm

The current chunker operates on each extracted page independently. It collapses runs of whitespace, trims the page, cuts passages no longer than the configured size (default 1,200 characters), prefers a nearby word boundary when possible, and starts the next passage before the prior passage ends by the configured overlap (default 180 characters). This provides neighboring context while preserving page attribution.

```
for each extracted page:
    normalize whitespace and trim
    start = 0
    while start is before page end:
        end = min(start + chunk_size, page_length)
        move end to a nearby word boundary when practical
        emit page_text[start:end] with original page number
        start = end - overlap, advanced to a word boundary
```

Chunk size and overlap are environment-configurable and validated so overlap remains smaller than size.

4.6 Local retrieval algorithm

Local mode seeds two clearly marked, illustrative exam-date and fee-deadline FAQ excerpts into the local JSON store. Their sample dates (18 November 2026 and 15 December 2026) are fictional test values, labeled “NOT AN OFFICIAL UNIVERSITY NOTICE,” and carry DEMO_FAQ_..._NOT_OFFICIAL.txt citations; they must be replaced by verified notices before real use. The matcher tokenizes the question and each chunk, removes common stop words, and applies simple plural normalization. It computes lexical coverage as the fraction of distinct question terms present in a chunk, sorts matches by descending coverage, and returns up to five. Chunks need at least one query-term match; the answer layer requires distance at most 0.75 and returns up to three matching excerpts verbatim with filename/page citations. Standalone greetings receive a brief welcome; if a non-greeting has no adequate match, local mode states that it cannot find evidence and offers sample questions without attaching fabricated citations. This deliberately simple algorithm is not a vector embedding and can miss paraphrases. Services-mode retrieval and its stricter abstention behavior are unchanged.

Chapter 5 — Implementation

5.1 Frontend

The Vite/React client provides a registration/login screen, a responsive application shell, suggested questions, a message composer, loading feedback while the API responds, source chips, conversation navigation, and thumbs-up/down controls. Administrators receive a knowledge-base view for document upload, listing, and deletion. API requests include credentials so the browser sends the HTTP-only session cookie.

Principal files: frontend/src/App.jsx, frontend/src/api.js, frontend/src/main.jsx, and frontend/src/styles.css.

5.2 Backend and API

Express configures Helmet, credentialed CORS for the configured frontend origin, JSON request size limits, cookie parsing, request logging outside test mode, authentication rate limits, and a centralized JSON error handler. Zod validates request payloads and environment values. The API has explicit authentication and role middleware.

Method	Endpoint	Access	Function
POST	/api/auth/register	Public; rate-limited	Create student account and session
POST	/api/auth/login	Public; rate-limited	Verify credentials and issue session
POST	/api/auth/logout	Public	Clear session cookie
GET	/api/auth/me	Authenticated	Return current user
POST	/api/chat	Authenticated; rate-limited	Retrieve evidence, answer, and persist turn
GET	/api/chat/conversations	Authenticated	List up to 50 conversations for current user
GET	/api/chat/conversations/:id	Authenticated	Fetch one owned conversation
DELETE	/api/chat/conversations/:id	Authenticated	Delete one owned conversation
POST	/api/chat/feedback	Authenticated	Record thumbs-up/down on assistant message
GET	/api/documents	Admin	List indexed documents
POST	/api/documents	Admin	Upload one PDF, DOCX, or TXT file
DELETE	/api/documents/:id	Admin	Remove metadata and indexed chunks
GET	/api/health	Public	Basic process health response

Principal files: backend/src/app.js, backend/src/index.js, backend/src/routes/, backend/src/middleware/authenticate.js, and backend/src/data/repository.js.

5.3 Authentication and authorization

Passwords are hashed with bcryptjs (12 rounds in application registration and administrator bootstrap). A signed JWT is placed in an HTTP-only cookie with SameSite=Lax; production sets the secure-cookie attribute. Session verification reloads the account and role from the selected repository. The document router requires the admin role. Conversation lookups include the authenticated user's ID, so one student cannot fetch or delete another user's conversation by changing an identifier.

Public registration has no option to specify an administrator role. The admin bootstrap script validates its input and creates an administrator in the configured data mode. The initial admin password is supplied as a command argument; operators should use a sufficiently strong unique password and avoid shell-history exposure on shared systems.

5.4 Document ingestion

The API accepts one in-memory multipart file per upload and enforces the MAX_UPLOAD_MB limit (default 15 MB, configurable up to 50 MB). Supported extensions are .pdf, .docx, and .txt. PDFs are limited to 500 pages; PDF.js extracts text page by page. Mammoth extracts raw DOCX text, and TXT input must be valid UTF-8. DOCX/TXT sources use page 1 because their extracted formats do not provide reliable printed page mapping. Scanned/image-only PDFs have no OCR support and fail when no readable text is found.

After successful extraction and chunking, the content is indexed and document metadata is recorded. In the services mode, partial Chroma indexing is rolled back on indexing failure, and vector entries are removed if the metadata record cannot be created.

5.5 Answer generation

Local mode

No external answer provider is called. Relevant chunk excerpts are quoted verbatim with source citations. This is useful for validating upload, search, chat, and citation flows, but it does not synthesize a natural-language answer.

Services mode

The embeddings service sends input in batches of at most 64 texts. ChromaDB stores vectors using cosine distance alongside the chunk text and metadata. The query is embedded and up to five nearest chunks are retrieved; results beyond RAG_MAX_DISTANCE (default 0.45) are excluded. If none remain, the API returns an abstention without calling the LLM. Otherwise, the Anthropic system prompt says to use only supplied excerpts, treat them as untrusted reference data, ignore instructions inside them, avoid guessing, and abstain if evidence is insufficient. Citations in the API response are deduplicated from retrieved document/page metadata.

5.6 Error handling and operational controls

Validation failures, unauthenticated access, role violations, missing records, upload limits, unavailable vector search, provider errors, and server failures return explicit HTTP status codes with JSON error messages. The application applies rate limits to authentication and chat routes. Startup validates required configuration and fails clearly when services-mode MongoDB cannot be reached. In local mode, the API prints a LOCAL DEVELOPMENT MODE banner; JSON writes are serialized, protected by a lock file for multiple local processes, and committed through temporary-file rename.

Chapter 6 — Testing and Results

6.1 Automated test coverage

The backend's Node.js test suite checks:

- Chunk-size bounds, page attribution, overlap, whitespace normalization, and invalid chunk parameters.
- Development JWT-secret generation, preservation of .env settings, placeholder replacement, refusal to generate outside development, and production rejection of weak configuration.
- End-to-end local-mode registration/login, student/admin authorization, TXT upload and indexing, question retrieval and page citation, conversation history, feedback persistence, and document/conversation deletion.
- Persistence across store instances and concurrent writes.

The suite is run from the repository root:

```
npm test
```

6.2 Verification performed

Check	Result
Backend automated tests	14 tests passed in the verified implementation run.
Frontend production build	npm run build completed successfully.
Local API and web startup	npm run dev started without Docker/MongoDB/ChromaDB; /api/health and Vite returned HTTP 200.
Local-mode integration	Registration, role checks, admin TXT ingestion/list/delete, cited answer, conversation history, feedback, and deletion exercised by automated integration test.
Production guard	Configuration test confirms NODE_ENV=production with DATA_MODE=local is rejected.
Supplied synopsis extraction	PDF text was extractable as four pages during project setup.
Live services-mode RAG	Not verified against live MongoDB, ChromaDB, embedding, or Anthropic services in this environment.

The test suite is functional verification, not a statistical answer-quality study. No precision, recall, latency, throughput, user-study, or hallucination-rate figures are claimed. Final evaluation should use approved university documents and a held-out set of representative student questions.

6.3 Suggested manual acceptance tests

1. Register a student and confirm that the admin panel is not exposed.
2. Sign in as the provisioned administrator and upload a short, searchable notice.
3. Ask a question whose answer is explicitly present; inspect the filename/page citation.
4. Ask about a fact absent from the corpus; confirm the assistant abstains.
5. Submit feedback, reload conversation history, and confirm it remains.
6. Delete the source as an administrator and verify its chunks no longer appear in retrieval.
7. In services mode, repeat with configured infrastructure and check that missing/low-similarity evidence yields abstention.

Chapter 7 — Setup and Deployment

7.1 Docker-free local development (default)

From the project root in Windows PowerShell:

```
npm install
npm run dev
```

Open http://localhost:5173; the API health endpoint is http://localhost:4000/api/health. The root development launcher waits for backend health before starting Vite; Vite proxies /api/* to http://localhost:4000 by default and forwards browser cookies and backend Set-Cookie headers unchanged. On first development startup, a strong random JWT secret is generated in the ignored root .env. Local application data is kept in the ignored .local-dev/data.json file and persists across restarts, including clearly labeled illustrative (not official) exam and fee FAQ excerpts for retrieval/citation tests. No Docker Desktop, database service, or AI credentials are required. Do not present the example dates as university information.

To create an administrator for this local data store, run in a second PowerShell terminal:

```
npm run create-admin --workspace backend -- admin@example.edu "use-a-unique-password-at-least-12-characters" "Campus Administrator"
```

Change the example email, password, and name. Do not use the example password for a real system. Local mode is for development and demonstrations only; back up or reset its data only when the app is stopped.

7.2 Services mode

Set DATA_MODE=services in the root .env, configure MongoDB, ChromaDB, a strong JWT_SECRET, OPENAI_API_KEY, and ANTHROPIC_API_KEY, then start local services and the app:

```
docker compose up -d mongodb chroma
npm run dev
```

In a second terminal, create an administrator as above. Docker Compose is optional when MongoDB and ChromaDB are already running at the configured URLs.

7.3 Production deployment

Production requires NODE_ENV=production, DATA_MODE=services, MongoDB, ChromaDB, provider credentials, a strong secret from a deployment secret manager, the exact HTTPS CLIENT_ORIGIN, and COOKIE_SECURE=true. Production rejects local mode and does not create JWT secrets. Build the client using npm run build; serve frontend/dist behind HTTPS and run the API with npm run start --workspace backend. The API binds to 0.0.0.0 and uses the platform-provided PORT (default 4000); the hosting ingress must forward /api/* to that process. By default, frontend requests use same-origin /api paths, preserving the HTTP-only session cookie and avoiding a browser request to a user's localhost. For example, an Nginx location /api/ may proxy to the backend on 127.0.0.1:4000. If the API is on another same-site origin, set VITE_API_URL to its origin (without /api) at client build time and set CLIENT_ORIGIN to the frontend origin. A direct POST /api/auth/login smoke test should return 200 and set the campus_session cookie for valid credentials; a browser CORS error indicates the exact CLIENT_ORIGIN or ingress preflight routing needs correction. Use private service networking, authenticated database deployments, secret management, backups, log monitoring, and a tested recovery procedure.

The supplied docker-compose.yml is for local development; it does not set up production TLS, MongoDB authentication, or public-service hardening. Do not expose its development ports publicly.

Chapter 8 — Limitations and Future Work

8.1 Current limitations

- Local keyword matching is lexical, not semantic; paraphrases and synonyms may not match. Local-mode quotations are evidence excerpts, not model-generated answers.
- Services-mode quality depends on source coverage, text extraction, chunk boundaries, retrieval threshold, embedding model, model behavior, and prompt configuration.
- Scanned PDFs are not OCR-processed; DOCX and TXT citations use page 1.
- The implementation has not been evaluated on a production corpus or under concurrent production load.
- Local JSON storage is a development convenience, not a substitute for a managed database, backup policy, or operational access controls.
- Authentication is application-managed; institutional single sign-on and student email-domain restrictions are not implemented.
- Current language processing is primarily English; multilingual support is not verified.

8.2 Future enhancements

1. Add OCR for scanned notices with page-level OCR confidence and review.
2. Evaluate retrieval and answer faithfulness against a curated question/evidence dataset.
3. Add source versioning, expiry/archival workflows, and duplicate-document detection.
4. Add institutional SSO and policy-based account provisioning.
5. Improve multilingual and Hindi/Hinglish query handling.
6. Add notification subscriptions for new examination notices and circulars.
7. Add production observability, audit trails, automated backups, and load/security testing.
8. Explore voice input and a mobile client after accessibility and privacy review.

Chapter 9 — Conclusion

The Smart Campus Assistant implements a full-stack foundation for asking questions against university documents. It combines a responsive React client, an authenticated Express API, role-separated administration, format-specific extraction, overlapping page-aware chunks, citations, chat persistence, and feedback. The services configuration provides the intended MongoDB, ChromaDB, embedding, and context-limited LLM pipeline. The default local mode makes the project demonstrable without Docker or paid external APIs while remaining clearly separate from production.

The core design goal is verifiability: an answer should be traceable to a supplied university document, and absence of evidence should lead to abstention rather than confident invention. Before institutional deployment, the system needs approved and current documents, production-grade service configuration, privacy/security review, and measured evaluation using representative campus questions.

References

[1] P. Lewis et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” Advances in Neural Information Processing Systems (NeurIPS), 2020. https://arxiv.org/abs/2005.11401

[2] React Documentation, https://react.dev/ [3] Vite Documentation, https://vite.dev/ [4] Express.js Documentation, https://expressjs.com/ [5] Mongoose Documentation, https://mongoosejs.com/docs/ [6] Chroma Documentation, https://docs.trychroma.com/ [7] Anthropic API Documentation, https://docs.anthropic.com/ [8] OpenAI API — Embeddings Guide, https://platform.openai.com/docs/guides/embeddings [9] Mozilla PDF.js, https://mozilla.github.io/pdf.js/ [10] Mammoth.js Documentation, https://github.com/mwilliamson/mammoth.js [11] Synopsis: Smart Campus Assistant: A RAG-Based Chatbot for University Information, VII Semester B.Tech project synopsis supplied with this project (institution/student/guide details to be verified before submission).

Appendix A — Configuration reference

Variable	Purpose
NODE_ENV	Runtime environment; production is strict about service mode and explicit secrets.
DATA_MODE	local for Docker-free development; services for MongoDB/ ChromaDB and AI APIs.
JWT_SECRET	Random secret of at least 43 characters and adequate character variety; never commit it.
MONGODB_URI	Application-data MongoDB connection string in services mode.
CHROMA_URL, CHROMA_COLLECTION	Vector database endpoint and collection.
OPENAI_API_KEY, OPENAI_BASE_URL, EMBEDDING_MODEL	Embedding service configuration.
ANTHROPIC_API_KEY, ANTHROPIC_MODEL	Answer-generation service configuration.
CHUNK_SIZE, CHUNK_OVERLAP	Text chunking parameters.
RAG_MAX_DISTANCE	Maximum Chroma cosine distance accepted for context.
MAX_UPLOAD_MB	Maximum upload size; default 15 MB.
CLIENT_ORIGIN, COOKIE_SECURE, JWT_EXPIRES_IN	Browser origin, cookie security, and session duration.

Do not include real secrets in a submitted report. Provide .env.example or a redacted configuration only.

Appendix B — Submission fields to review

- [ ] Verify institution, campus, department, programme, semester, section, academic year, and submission date.
- [ ] Verify student name and enrollment number against current institutional records.
- [ ] Verify project guide name, designation, and spelling.
- [ ] Complete certificate and declaration fields; obtain required signatures and institutional seal.
- [ ] Edit acknowledgement to reflect actual contributions.
- [ ] Add institution-approved screenshots, diagrams, and page formatting requirements if requested.
- [ ] Confirm final test results and replace any environment-specific statement if new testing is performed.
- [ ] Follow the institution's required citation style, originality rules, and report template.